

LangChain in Python — A Detailed, Practical Tutorial

Building LLM Applications with LCEL, Gemini, and Ollama

This is a hands-on, in-depth guide to **LangChain**, the most widely used framework for building applications on top of large language models. It assumes you can write Python and have made a basic API call before, but have never touched LangChain or LLM orchestration.

Every concept is taught three ways: **what it is**, **what is actually happening under the hood**, and **working code** against two backends so you are never locked to one provider:

- **Google Gemini** — a hosted frontier model, accessed with an API key.
- **Ollama** — open-weight models (Llama 3.1, Qwen, and others) running locally on your own machine: free, private, offline-capable.

All code uses **LangChain 1.0+** conventions — **LCEL** (LangChain Expression Language) for composition and the LangGraph-based `create_agent` for agents. Deprecated constructs (`LLMChain` , `ConversationChain` , `initialize_agent`) appear only as labelled historical notes so you can recognise them in old code and avoid them.

How to read this guide. Each chapter builds on the previous one. Type the code out rather than copy-pasting — the muscle memory of LCEL's pipe syntax is most of the learning. Interview questions at the end of each chapter double as self-tests.

1. Introduction & Setup

1.1 Why LangChain exists

A direct call to an LLM API is a single function call: you send text, you get text back. That is enough for a one-shot demo and nothing more. Real applications need an entire supporting cast around the model, and writing that plumbing by hand — for every provider, every project — is where the time goes. LangChain exists to provide that plumbing as reusable, composable parts.

Concretely, LangChain gives you:

- **A provider-agnostic interface.** Every chat model — Gemini, Ollama, OpenAI, Anthropic — exposes the *same* methods: `.invoke()` , `.stream()` , `.batch()` , and async variants. You can switch providers by changing one line, which matters enormously when a cheaper or better model appears next quarter.
- **Composability via LCEL.** Prompts, models, parsers, retrievers, and arbitrary Python functions are all **Runnables** that snap together with the `|` operator into pipelines.
- **Conversational memory.** Helpers that replay prior turns into each request so a chatbot remembers context.
- **Retrieval-Augmented Generation (RAG).** A full toolkit — loaders, splitters, embeddings, vector stores, retrievers — for grounding answers in your own documents.
- **Agents and tools.** A reasoning loop in which the model itself decides which tools to call, in what order, to accomplish a goal.
- **Production tooling.** Tracing, evaluation, structured output, retries, and fallbacks for when the prototype has to become a service.

A useful mental model: **the LLM is the engine; LangChain is the rest of the car** — the transmission, steering, and dashboard that turn raw power into something you can drive.

1.2 The package ecosystem

Modern LangChain is deliberately split into many small packages so the core stays lightweight and each integration can version independently. You install only what you use.

Package	Provides
<code>langchain-core</code>	The <code>Runnable</code> interface, prompts, output parsers, message types — the foundation everything else builds on.
<code>langchain</code>	High-level constructs such as <code>create_agent</code> , plus convenience re-exports.
<code>langchain-google-genai</code>	The Gemini integration (<code>ChatGoogleGenerativeAI</code> , <code>GoogleGenerativeAIEmbeddings</code>).
<code>langchain-ollama</code>	The local-model integration (<code>ChatOllama</code> , <code>OllamaEmbeddings</code>).
<code>langchain-community</code>	Community-maintained integrations, including most document loaders.
<code>langchain-text-splitters</code>	Chunking utilities for RAG.
<code>langchain-chroma</code>	The Chroma vector store integration.

1.3 Installation

Create an isolated virtual environment first — never install into the system Python.

```
# Create and activate a virtual environment (Python 3.10+ recommended)
python -m venv .venv
source .venv/bin/activate          # Windows: .venv\Scripts\activate

# Core framework + LCEL runtime
pip install langchain langchain-core

# Model providers
pip install langchain-google-genai  # Google Gemini (hosted)
pip install langchain-ollama        # Ollama (local models)

# RAG building blocks
pip install langchain-community langchain-text-splitters langchain-chroma pypdf

# Agents
pip install langgraph

# Convenience for loading secrets in development
pip install python-dotenv
```

Verify the install:

```
import langchain
print(langchain.__version__)  # expect 1.x
```

Best practice: Pin versions in a `requirements.txt` (e.g. `langchain==1.0.x`). LangChain moves quickly; an unpinned environment can break between `pip install` runs.

1.4 Configuring Gemini

Create a free API key in Google AI Studio. The Gemini integration reads the `GOOGLE_API_KEY` environment variable automatically — you should not pass keys as literals in code.

```
# macOS / Linux (current shell)
export GOOGLE_API_KEY="your-gemini-api-key-here"
```

For repeatable local development, keep secrets in a `.env` file and load them:

```
# .env (add this file to .gitignore — never commit secrets)
# GOOGLE_API_KEY=your-gemini-api-key-here

from dotenv import load_dotenv
load_dotenv()  # reads .env and populates os.environ before any model is created
```

1.5 Configuring Ollama

Ollama runs models on your own hardware, so there is **no API key and no per-token cost**. Install the Ollama application (which starts a local server on `http://localhost:11434`), then pull the models this tutorial uses:

```
ollama pull llama3.1          # general chat / reasoning / tool-calling model
ollama pull nomic-embed-text  # embedding model used in the RAG chapter

# Sanity check: list installed models and run a quick prompt
ollama list
ollama run llama3.1 "Say hello in five words."
```

Keep the Ollama server running while you work through the examples.

Hosted vs local — how to choose. Use **Gemini** when you want top-tier quality with zero infrastructure and are comfortable sending data to a cloud API. Use **Ollama** when you need privacy, offline operation, predictable zero marginal cost, or freedom to experiment without a metered bill. Because LangChain abstracts the model, you can prototype on one and deploy on the other.

Interview Questions — Setup

1. **What does LangChain provide that a raw LLM API call does not?** *(Orchestration: provider-agnostic interface, composition, memory, retrieval, agents/tools, and production tooling like tracing and structured output.)*
2. **Why is LangChain split into many packages such as `langchain-core` and `langchain-google-genai`?** *(To keep the core lightweight, let integrations version independently, and minimise dependency bloat — you install only what you use.)*
3. **What is the role of `langchain-core` specifically?** *(It defines the foundational abstractions — the Runnable interface, prompts, parsers, message types — that every other package depends on.)*
4. **Operationally, how does serving a model with Ollama differ from Gemini?** *(Ollama runs locally with no key, no per-token cost, and data stays on-device; Gemini is a hosted API needing a key and network, billed per token.)*
5. **Why should API keys be read from environment variables rather than written in code?** *(To avoid committing secrets to version control and to allow different keys per environment without code changes.)*

2. Core Concept 1 — Models & Prompts

2.1 Chat models and the message abstraction

Today's LLMs are **chat models**: their native input is not a single string but an ordered list of **messages**, each tagged with a role. LangChain models this with three primary message types:

- **SystemMessage** — standing instructions that set behaviour ("You are a terse SQL expert").
- **HumanMessage** — input from the user.
- **AIMessage** — a response produced by the model (and, in conversations, replayed back as history).

When you call `.invoke("some text")` with a bare string, LangChain quietly wraps it in a single **HumanMessage** for you. Being explicit is often clearer:

```
from langchain_core.messages import SystemMessage, HumanMessage
from langchain_google_genai import ChatGoogleGenerativeAI

model = ChatGoogleGenerativeAI(model="gemini-2.5-flash")

messages = [
    SystemMessage(content="You are a concise assistant. Answer in one sentence."),
    HumanMessage(content="What is a transformer in machine learning?"),
]

response = model.invoke(messages)
print(response.content)
```

2.2 Initialising a model and its key parameters

```
from langchain_google_genai import ChatGoogleGenerativeAI

model = ChatGoogleGenerativeAI(
    model="gemini-2.5-flash", # which model variant to call
    temperature=0.7,         # 0 = deterministic, higher = more random/creative
    max_tokens=512,          # cap on the length of the generated response
    timeout=30,              # seconds to wait before giving up
    max_retries=2,           # automatic retries on transient errors
)
```

The same parameters apply to Ollama — only the import and model name change:

```
from langchain_ollama import ChatOllama

model = ChatOllama(
    model="llama3.1",
    temperature=0.7,
    num_predict=512,  # Ollama's equivalent of max_tokens
)
```

This symmetry is the heart of LangChain: **downstream code never needs to know which provider it is talking to.**

Note: Some newer Gemini models default `temperature` to `1.0` and may behave poorly at very low values. If you need determinism, set `temperature=0` explicitly and test the result.

2.3 Anatomy of an `AIMessage`

`.invoke()` does not return a string — it returns an `AIMessage` object rich with metadata you will use constantly in production.

```
response = model.invoke("Name the capital of France.")

print(response.content)           # "Paris" — the text you usually want
print(response.response_metadata) # finish reason, safety ratings, model name, etc.
print(response.usage_metadata)    # {'input_tokens': ..., 'output_tokens': ..., 'total_tokens': ...}
print(response.tool_calls)        # populated when the model decides to call a tool (see ch. 6)
```

`usage_metadata` is how you track and bill token cost; `tool_calls` is the mechanism that makes agents possible. Remembering that `.content` is just one attribute of a larger object will save you confusion later.

2.4 Running modes: invoke, stream, batch

Every chat model supports four ways of being called, and you get all of them for free:

```
# 1. invoke — single input, single AIMessage out (blocking)
model.invoke("Define entropy.")

# 2. stream — yields partial chunks as the model generates them (great for UX)
for chunk in model.stream("Write a haiku about databases."):
    print(chunk.content, end="", flush=True)

# 3. batch — many inputs processed concurrently (great for throughput)
model.batch(["Capital of Japan?", "Capital of Peru?", "Capital of Kenya?"])

# 4. async — ainvoke / astream / abatch for use inside async applications
# result = await model.ainvoke("Explain async briefly.")
```

Streaming matters more than it looks: an eight-second wait on a blank screen feels broken, whereas tokens appearing immediately feels instant even if total time is identical.

2.5 Prompt templates

Hardcoding prompts as f-strings scatters wording through your codebase and mixes structure with data. A **prompt template** separates the two and plugs straight into LCEL chains.

`ChatPromptTemplate` is the modern default — it preserves message roles:

```

from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are an expert {domain} tutor. Answer in {style} style."),
    ("human", "Explain: {question}"),
])

# .invoke() fills the {placeholders} and returns a list of formatted messages.
messages = prompt.invoke({
    "domain": "physics",
    "style": "ELI5",
    "question": "why is the sky blue?",
})

print(messages.to_messages())

```

PromptTemplate produces a single plain string and suits older completion-style models:

```

from langchain_core.prompts import PromptTemplate

template = PromptTemplate.from_template(
    "Summarise the following text in {n} bullet points:\n\n{text}"
)

print(template.format(n=3, text="LangChain is a framework for ..."))

```

2.6 Partial prompts and few-shot prompting

You can **partially** fill a template — useful for values known at construction time, such as a date or a fixed format instruction:

```

partial = prompt.partial(style="formal")
# now only {domain} and {question} remain to be supplied at call time

```

For tasks where examples teach the model the desired pattern, use **few-shot** templates:

```

from langchain_core.prompts import FewShotChatMessagePromptTemplate, ChatPromptTemplate

examples = [
    {"input": "2 + 2", "output": "4"},
    {"input": "5 - 3", "output": "2"},
]

example_prompt = ChatPromptTemplate.from_messages([
    ("human", "{input}"),
    ("ai", "{output}"),
])

few_shot = FewShotChatMessagePromptTemplate(
    examples=examples,
    example_prompt=example_prompt,
)

final_prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a calculator that only outputs the number."),
    few_shot,  # the worked examples are injected here
    ("human", "{input}"),
])

```

Interview Questions — Models & Prompts

1. **What are the three core message types in LangChain and what does each represent?** (*SystemMessage* for standing instructions, *HumanMessage* for user input, *AIMessage* for model output / replayed history.)
2. **.invoke() returns an AIMessage, not a string. Name two useful attributes besides .content .** (*usage_metadata* for token counts/cost, *tool_calls* for agent tool invocations; also *response_metadata* for finish reason and safety info.)
3. **When would you choose .stream() over .invoke() ?** (For user-facing latency: streaming shows tokens immediately, dramatically improving perceived responsiveness even when total time is unchanged.)

4. **What advantage does `.batch()` give you?** (Concurrent processing of many inputs for higher throughput compared with sequential `invoke` calls.)
5. **Why prefer `ChatPromptTemplate` over a Python f-string?** (Reusability, separation of structure from data, role awareness, partial filling, and direct composability into LCEL chains.)
6. **What does `.partial()` do on a prompt template, and when is it useful?** (Pre-fills some variables at construction time so only the rest are supplied at call time — handy for fixed values like dates or format instructions.)
7. **What problem does few-shot prompting solve?** (It steers output format and style by showing the model worked examples, improving consistency on structured or pattern-based tasks without fine-tuning.)

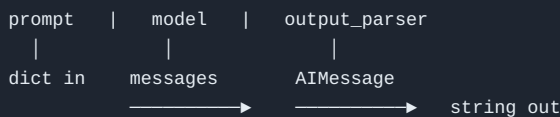
3. Core Concept 2 — LCEL (The Core Engine)

3.1 Everything is a Runnable

LCEL rests on a single abstraction: the **Runnable**. A Runnable is any object that implements a common interface — `invoke`, `stream`, `batch`, and their async forms. Prompt templates, chat models, output parsers, retrievers, and even plain Python functions can all behave as Runnables. Because they share an interface, they can be **composed**, and the composition is itself a Runnable.

3.2 The pipe operator, explained

LangChain overrides Python's `|` operator (the `__or__` dunder method) on Runnables so that `a | b` returns a **RunnableSequence** — a new Runnable that, when invoked, feeds the output of `a` into `b`. It reads left to right exactly like a Unix shell pipe:



This is not magic syntax bolted on; it is plain operator overloading. The payoff is that the assembled chain transparently inherits `invoke`, `stream`, `batch`, `async`, and automatic LangSmith tracing.

3.3 Your first chain: prompt → model → parser

An **output parser** turns the model's `AIMessage` into something convenient. `StrOutputParser` simply extracts `.content`.

Gemini:

```
from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.output_parsers import StrOutputParser

model = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0)
prompt = ChatPromptTemplate.from_messages([
    ("system", "You are an expert {domain} engineer."),
    ("human", "Answer in one paragraph: {question}"),
])
parser = StrOutputParser()

# Compose the three Runnables into a single pipeline.
chain = prompt | model | parser

# One call runs all three stages. The dict fills the prompt's placeholders.
print(chain.invoke({"domain": "data", "question": "What is a vector index?"}))
```

Ollama — only the model line changes; prompt and parser are reused verbatim:

```
from langchain_ollama import ChatOllama

model = ChatOllama(model="llama3.1", temperature=0)
chain = prompt | model | parser
print(chain.invoke({"domain": "data", "question": "What is a vector index?"}))
```

3.4 The chain inherits every running mode

Because `chain` is a Runnable, it streams, batches, and runs async with no extra code:

```
# Stream tokens of the final string as they arrive
for token in chain.stream({"domain": "ML", "question": "What is a GPU?"}):
    print(token, end="", flush=True)

# Process several inputs concurrently
chain.batch([
    {"domain": "ML", "question": "What is overfitting?"},
    {"domain": "ML", "question": "What is regularisation?"},
])
```

3.5 The structural Runnables: Parallel, Passthrough, Lambda

Real chains are rarely a straight line. Three primitives let you branch, forward, and inject custom logic.

`RunnableParallel` runs several Runnables on the *same* input concurrently and returns a dict of their results. You can write it as a plain dict — `LangChain` coerces it automatically:

```
from langchain_core.runnables import RunnableParallel

# These two sub-chains both receive the original input and run in parallel.
parallel = RunnableParallel(
    joke=(ChatPromptTemplate.from_template("Tell a joke about {topic}") | model | parser),
    fact=(ChatPromptTemplate.from_template("State one fact about {topic}") | model | parser),
)
print(parallel.invoke({"topic": "databases"}))
# -> {"joke": "...", "fact": "..."}


```

`RunnablePassthrough` forwards its input unchanged — invaluable when one branch needs the raw input while another transforms it:

```
from langchain_core.runnables import RunnablePassthrough

# .assign() adds new keys to a dict while keeping the originals intact.
enrich = RunnablePassthrough.assign(
    length=lambda x: len(x["text"])
)
print(enrich.invoke({"text": "hello"})) # -> {"text": "hello", "length": 5}


```

`RunnableLambda` wraps an arbitrary Python function as a Runnable so it can sit inside a chain. Plain functions are coerced automatically when piped:

```
from langchain_core.runnables import RunnableLambda

def shout(text: str) -> str:
    return text.upper() + "!"

chain = prompt | model | parser | RunnableLambda(shout)
# or simply: prompt | model | parser | shout


```

These three — parallel, passthrough, lambda — are the joints that let you build arbitrarily shaped graphs while staying inside LCEL. You will use all three in the RAG chapter.

3.6 Configuring and hardening chains

LCEL Runnables expose composable configuration methods:

```
# Bind fixed call-time arguments (e.g. stop sequences) to a model
model_with_stop = model.bind(stop=["\n\n"])

# Attach metadata for tracing / debugging
named = chain.with_config(run_name="vector-index-explainer", tags=["docs"])

# Provide a fallback chain if the primary one errors (e.g. provider outage)
gemini_chain = prompt | ChatGoogleGenerativeAI(model="gemini-2.5-flash") | parser
ollama_chain = prompt | ChatOllama(model="llama3.1") | parser
robust = gemini_chain.with_fallbacks([ollama_chain])

# Automatically retry transient failures with backoff
resilient = chain.with_retry(stop_after_attempt=3)
```

Historical context: Before LCEL you wrote `LLMChain(llm=model, prompt=prompt)`, which was deprecated in LangChain 0.1.17. It had no native streaming, batching, async, or fallbacks — all of which LCEL provides for free. Recognise `LLMChain` in legacy code, but never write it anew.

Interview Questions — LCEL

- What is a Runnable, and why is the concept central to LCEL?** (A common interface — *invoke/stream/batch/async* — implemented by prompts, models, parsers, and functions; shared interface is what makes them composable.)
- Mechanically, what does the `|` operator do?** (It overloads `__or__` to build a `RunnableSequence` in which each component's output becomes the next component's input.)
- Name three capabilities a composed chain inherits automatically.** (Streaming, batching, async; also tracing, fallbacks, and retries.)
- What is the difference between `RunnableParallel` and `RunnablePassthrough`?** (`RunnableParallel` runs multiple Runnables on the same input and returns a dict of results; `RunnablePassthrough` forwards input unchanged, with `.assign()` to add keys.)
- Why would you wrap a function in `RunnableLambda` (or pipe a bare function)?** (To insert custom Python logic as a first-class step inside an LCEL chain.)
- How would you make a chain resilient to a provider outage?** (Use `.with_fallbacks([...])` to fall back to another model/chain, and `.with_retry()` for transient errors.)
- Why is LCEL preferred over the legacy `LLMChain`?** (Less boilerplate, native streaming/async/batch, composability, fallbacks/retries, and first-class observability.)
- What does `StrOutputParser` return and why is it commonly the last step?** (It extracts the model's text content as a plain string, the form most application code wants.)

4. Core Concept 3 — Memory

4.1 Why memory is necessary

An LLM is **stateless**: each `.invoke()` is a fresh, independent request with no recollection of earlier calls. The model never "remembers" anything — the *only* way it can appear to remember is if you resend the prior conversation as part of every new request. Memory in LangChain is therefore not storage *inside* the model; it is the machinery that **loads past messages and replays them into the prompt** on each turn, then **saves the new exchange** for next time.

This has an important consequence: conversation history consumes context-window tokens and grows every turn, so long chats get slower and more expensive. Managing that growth (trimming or summarising) is part of doing memory well.

4.2 The modern pattern: `RunnableWithMessageHistory`

`RunnableWithMessageHistory` wraps any chain and automates the load-replay-save cycle, keyed by a **session ID** so different users' conversations stay separate.

Step 1 — reserve a slot for history in the prompt using `MessagesPlaceholder` :


```

from langchain_core.prompts import ChatPromptTemplate, MessagesPlaceholder

prompt = ChatPromptTemplate.from_messages([
    ("system", "You are a helpful assistant. Be concise."),
    MessagesPlaceholder(variable_name="history"), # prior turns injected here
    ("human", "{input}"),
])

```

Step 2 — build the chain and wrap it with history:

```

from langchain_google_genai import ChatGoogleGenerativeAI
from langchain_core.output_parsers import StrOutputParser
from langchain_core.chat_history import InMemoryChatMessageHistory
from langchain_core.runnables.history import RunnableWithMessageHistory

model = ChatGoogleGenerativeAI(model="gemini-2.5-flash")
chain = prompt | model | StrOutputParser()

# A store mapping session_id -> that session's message history.
store = {}

def get_session_history(session_id: str) -> InMemoryChatMessageHistory:
    if session_id not in store:
        store[session_id] = InMemoryChatMessageHistory()
    return store[session_id]

conversational = RunnableWithMessageHistory(
    chain,
    get_session_history,
    input_messages_key="input", # which input key is the user's new message
    history_messages_key="history", # must match the MessagesPlaceholder name
)

```

Step 3 — converse, identifying the session via `config`:

```

cfg = {"configurable": {"session_id": "user-123"}}

print(conversational.invoke({"input": "Hi, I'm Alex and I work in robotics."}, config=cfg))
print(conversational.invoke({"input": "What field do I work in?"}, config=cfg))
# -> correctly answers "robotics" because the first turn was replayed

```

Switching to **Ollama** changes only the model line; the entire memory wrapper is identical:

```

from langchain_ollama import ChatOllama
model = ChatOllama(model="llama3.1")
chain = prompt | model | StrOutputParser()
# ... wrap with RunnableWithMessageHistory exactly as above

```

4.3 Controlling history growth: trimming

Left unchecked, history eventually overflows the context window. `trim_messages` keeps the most recent messages within a token budget:

```

from langchain_core.messages import trim_messages

trimmer = trim_messages(
    max_tokens=2000,
    strategy="last", # keep the most recent messages
    token_counter=model, # use the model's tokeniser to count
    include_system=True, # always retain the system message
    start_on="human", # ensure the trimmed history starts cleanly
)

# Insert the trimmer before the model in your chain so history is pruned each turn.
chain = prompt | trimmer | model | StrOutputParser()

```

For very long conversations, an alternative is **summarisation memory**: periodically replace old turns with an LLM-generated summary, preserving gist while shrinking token count. The trade-off is fidelity — summaries lose detail.

4.4 Persistence in production

`InMemoryChatMessageHistory` lives in process memory and vanishes on restart; it also cannot be shared across multiple server instances. For real applications, swap it for a persistent history backend (Redis, Postgres, SQLite, MongoDB, etc.). The only thing that changes is the object returned by `get_session_history` — the chain and wrapper stay the same, which is the whole point of the abstraction.

Historical context: Older code uses `ConversationBufferMemory` with `ConversationChain`, or `ConversationSummaryMemory`. These are deprecated. For chains use `RunnableWithMessageHistory`; for agents use a **checkpointer** (chapter 6). Both scope state by an ID and integrate with persistent stores.

Interview Questions — Memory

1. **Why are LLMs called stateless, and where does "memory" actually live?** *(Each call is independent; memory lives outside the model — it replays stored prior messages into the prompt on every call.)*
2. **What does the `session_id` accomplish in `RunnableWithMessageHistory`?** *(It isolates and identifies separate conversations so different users' histories never mix.)*
3. **What must the `history_messages_key` match, and why?** *(The `variable_name` of the `MessagesPlaceholder`, so loaded history is injected into the correct slot in the prompt.)*
4. **Why does conversation history make long chats slower and costlier?** *(History is resent every turn and counts against the context window, so token usage grows with conversation length.)*
5. **Give two strategies for keeping history within the context window.** *(Trim to the most recent messages within a token budget via `trim_messages`; or summarise older turns into a compact summary.)*
6. **Why is `InMemoryChatMessageHistory` unsuitable for production, and what replaces it?** *(It is lost on restart and not shared across servers; use a persistent backend like Redis or Postgres.)*

5. Core Concept 4 — RAG (Retrieval-Augmented Generation)

5.1 What RAG is and why it beats fine-tuning for knowledge

LLMs only know what was in their training data, frozen at a cutoff date, and they cannot see your private documents. **RAG** fixes this by retrieving relevant snippets from *your* data at query time and inserting them into the prompt, so the model answers from facts you supply rather than memory alone.

Compared with fine-tuning, RAG is cheaper, updates instantly (just change the documents), and lets answers cite their sources. Fine-tuning changes *behaviour and style*; RAG supplies *knowledge*. For "answer questions about my documents," RAG is almost always the right tool.

The pipeline has two phases:

```
INDEXING (once, offline):
    Load documents → Split into chunks → Embed each chunk → Store vectors in a DB

RETRIEVAL (per query):
    Embed question → Find nearest chunks → Insert into prompt → Model answers
```

5.2 Step 1 — Load documents

Document loaders turn a source (PDF, web page, CSV, Notion, etc.) into a list of `Document` objects, each carrying `page_content` (text) and `metadata` (source, page number).

```
from langchain_community.document_loaders import PyPDFLoader

loader = PyPDFLoader("employee_handbook.pdf")
documents = loader.load()           # one Document per page
print(len(documents), "pages")
print(documents[0].metadata)        # e.g. {'source': 'employee_handbook.pdf', 'page': 0}
```

5.3 Step 2 — Split into chunks

Whole documents are too large for the context window and too coarse for precise retrieval. The **text splitter** breaks them into overlapping chunks. `RecursiveCharacterTextSplitter` is the workhorse: it tries to split on paragraph breaks first, then sentences, then words, preserving natural boundaries.

```
from langchain_text_splitters import RecursiveCharacterTextSplitter

splitter = RecursiveCharacterTextSplitter(
    chunk_size=1000,    # target characters per chunk
    chunk_overlap=150,  # characters shared between neighbours, so context isn't cut mid-thought
    separators=["\n\n", "\n", ". ", " ", ""], # tried in order
)
chunks = splitter.split_documents(documents)
print(len(chunks), "chunks")
```

Tuning guidance: smaller chunks (300–500 chars) give precise retrieval but may lack context; larger chunks (1000–2000) give context but dilute relevance and cost more tokens. Overlap of 10–20% of chunk size is a sensible default.

5.4 Step 3 — Embed and store

An **embedding model** maps text to a fixed-length numeric vector positioned so that semantically similar texts sit close together (measured by cosine similarity). Retrieval is then "find the chunk vectors nearest the question vector." A **vector store** holds these vectors and does the nearest-neighbour search.

Gemini embeddings + Chroma:

```
from langchain_google_genai import GoogleGenerativeAIEmbeddings
from langchain_chroma import Chroma

embeddings = GoogleGenerativeAIEmbeddings(model="models/text-embedding-004")

vectorstore = Chroma.from_documents(
    documents=chunks,
    embedding=embeddings,
    persist_directory="./chroma_db", # write to disk so you can reload without re-indexing
)
```

Ollama embeddings + Chroma (fully local, no API cost):

```
from langchain_ollama import OllamaEmbeddings
from langchain_chroma import Chroma

embeddings = OllamaEmbeddings(model="nomic-embed-text")
vectorstore = Chroma.from_documents(chunks, embeddings, persist_directory="./chroma_db")
```

Critical rule: use the *same* embedding model for indexing and querying. Different models produce incompatible vector spaces, so similarity search silently returns garbage. **Chroma** is a great local default; **FAISS** is a fast in-memory alternative for larger corpora; hosted options (Pinecone, Weaviate) suit scale.

5.5 Step 4 — Retriever and the RAG chain

Turn the store into a **retriever** and assemble the LCEL chain. This is where chapter 3's structural Runnables earn their keep: `RunnableParallel` (written as a dict) builds the prompt inputs by running the retriever and forwarding the question simultaneously.

```

from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnablePassthrough
from langchain_core.output_parsers import StrOutputParser
from langchain_google_genai import ChatGoogleGenerativeAI

# Retriever: search strategy + how many chunks to return.
retriever = vectorstore.as_retriever(
    search_type="similarity",      # or "mmr" for diversity, "similarity_score_threshold"
    search_kwargs={"k": 4},       # return the 4 nearest chunks
)

model = ChatGoogleGenerativeAI(model="gemini-2.5-flash", temperature=0)

prompt = ChatPromptTemplate.from_template(
    """Answer the question using ONLY the context below.
    If the answer is not contained in the context, say "I don't know."

    Context:
    {context}

    Question: {question}
    Answer: """
)

# Helper: flatten retrieved Document objects into one context string.
def format_docs(docs):
    return "\n\n".join(d.page_content for d in docs)

# The RAG chain:
# - "context": retrieve chunks for the question, then format them
# - "question": pass the user's question straight through
rag_chain = (
    {"context": retriever | format_docs, "question": RunnablePassthrough()}
    | prompt
    | model
    | StrOutputParser()
)

print(rag_chain.invoke("What is the company's remote-work policy?"))

```

To run this **entirely locally**, swap in `ChatOllama(model="llama3.1")` and `OllamaEmbeddings(model="nomic-embed-text")`. The chain structure does not change at all.

5.6 Returning sources, and conversational RAG

Production RAG usually needs to **cite sources**. Use `RunnablePassthrough.assign` to keep the retrieved documents alongside the answer:

```

from langchain_core.runnables import RunnableParallel, RunnablePassthrough

rag_with_sources = RunnableParallel(
    context=retriever,
    question=RunnablePassthrough(),
).assign(
    answer=(
        lambda x: {"context": format_docs(x["context"]), "question": x["question"]}
    ) | prompt | model | StrOutputParser()
)

result = rag_with_sources.invoke("What is the remote-work policy?")
print(result["answer"])
for doc in result["context"]:
    print("Source:", doc.metadata)

```

For multi-turn RAG ("...and what about part-time staff?"), a follow-up may be meaningless out of context. The standard fix is a **history-aware retriever**: first ask the model to rewrite the follow-up into a standalone question using the chat history, then retrieve with that rewritten query. Conceptually it is one extra LCEL step (rephrase → retrieve → answer) layered on top of the chain above.

5.7 Search strategies worth knowing

- `similarity` — pure nearest-neighbour; the default.
- `mmr` (Maximal Marginal Relevance) — balances relevance against diversity so you don't get four near-identical chunks.
- `similarity_score_threshold` — return only chunks above a confidence floor, so weak matches are dropped rather than padded.

Interview Questions — RAG

1. **What does RAG solve that fine-tuning does not?** *(Injecting current or private knowledge at query time without retraining, with the ability to cite sources; fine-tuning changes behaviour/style, not knowledge.)*
2. **Walk through the two phases of a RAG pipeline.** *(Indexing: load → split → embed → store. Retrieval: embed query → nearest-neighbour search → insert chunks into prompt → generate answer.)*
3. **Why split documents into overlapping chunks?** *(To fit the context window and sharpen retrieval; overlap prevents losing meaning at chunk boundaries.)*
4. **What is an embedding, and why must indexing and querying use the same embedding model?** *(A semantic vector representation of text; different models produce incompatible vector spaces, breaking similarity search.)*
5. **What does the retriever's `k` parameter control, and what is the trade-off?** *(How many chunks are returned; too few omits needed context, too many adds noise, cost, and can exceed the context window.)*
6. **In the LCEL RAG chain, what is the purpose of `RunnablePassthrough` ?** *(It forwards the original question unchanged while the parallel retriever branch builds the context.)*
7. **What is MMR retrieval and when would you use it?** *(Maximal Marginal Relevance balances relevance with diversity to avoid redundant, near-duplicate chunks.)*
8. **How would you support follow-up questions in a conversational RAG system?** *(Use a history-aware retriever that rewrites the follow-up into a standalone query using chat history before retrieving.)*

6. Core Concept 5 — Agents & Tools

6.1 Chains vs agents

A **chain** runs a fixed, predetermined sequence of steps. An **agent** is dynamic: you give the model a goal and a set of **tools**, and the model itself decides *which* tools to call, *with what arguments*, and *in what order*, looping until the task is solved. The defining feature is the **reasoning loop** — control flow is determined by the model at runtime, not hardcoded by you.

A concise way to think about it: **Agent = Model + Tools + a loop**. On each iteration the runtime rebuilds the full context (system prompt + tool definitions + message history so far) and asks the model for its next move. The model replies with either a final answer or one or more **tool calls**; if it calls tools, the runtime executes them, appends the results to the history, and loops again.

In LangChain 1.0 the entry point is `create_agent` from `langchain.agents`, built on **LangGraph**, which supplies the durable loop, streaming, memory, and human-in-the-loop machinery so you don't implement them yourself.

6.2 Defining tools

The `@tool` decorator turns a typed Python function into a tool. Two things matter enormously:

- **The docstring** — the model reads it to decide *when* the tool is relevant. Vague docstrings cause wrong or missed tool calls.
- **The type hints** — they define the tool's argument schema that the model must fill.

```

from langchain_core.tools import tool

@tool
def calculator(expression: str) -> str:
    """Evaluate a basic arithmetic expression, e.g. '23 * 17 + 4'.
    Use this for any math the user asks about."""
    try:
        # eval is used for brevity; in production use a safe math parser.
        return str(eval(expression))
    except Exception as e:
        return f"Error: {e}"

@tool
def get_word_length(word: str) -> int:
    """Return the number of letters in a single word."""
    return len(word)

```

For richer argument validation, attach a Pydantic schema so the model is told exactly what each field means:

```

from pydantic import BaseModel, Field
from langchain_core.tools import tool

class WeatherInput(BaseModel):
    city: str = Field(description="City name, e.g. 'Leeds'")
    units: str = Field(default="celsius", description="'celsius' or 'fahrenheit'")

@tool(args_schema=WeatherInput)
def get_weather(city: str, units: str = "celsius") -> str:
    """Return the current weather for a city."""
    return f"It's 18° {units} and cloudy in {city}."

```

6.3 Creating the agent

Configure the essentials directly: `model`, `tools`, and an optional `system_prompt`.

Gemini (the model may be passed as a `"provider:model"` string):

```

from langchain.agents import create_agent

agent = create_agent(
    model="google_genai:gemin-2.5-flash",
    tools=[calculator, get_word_length],
    system_prompt="You are a precise assistant. Use tools for math and counting.",
)

```

Ollama (pass an initialised, tool-capable model instance):

```

from langchain.agents import create_agent
from langchain_ollama import ChatOllama

ollama_model = ChatOllama(model="llama3.1") # llama3.1 supports tool calling
agent = create_agent(
    model=ollama_model,
    tools=[calculator, get_word_length],
    system_prompt="You are a precise assistant. Use tools for math and counting.",
)

```

6.4 Running the agent and inspecting its reasoning

Agents take a `messages` list and return the full message trace, so you can see every reasoning step and tool call.

```

result = agent.invoke({
    "messages": [
        {"role": "user",
         "content": "How many letters are in 'orchestration', and what is that times 8?"}
    ]
})

# The final answer is the content of the last message.
print(result["messages"][-1].content)

# Inspect the entire trace: user → tool call → tool result → ... → answer.
for m in result["messages"]:
    m.pretty_print()

```

Under the hood the agent: reads the question → emits a tool call `get_word_length("orchestration")` → receives `13` → emits `calculator("13 * 8")` → receives `104` → writes a natural-language final answer. **You wrote no branching logic;** the model orchestrated the tools itself.

6.5 Streaming intermediate steps

For responsive UIs, stream the agent's progress instead of waiting for the whole loop:

```

from langchain_core.messages import AIMessage, HumanMessage

for chunk in agent.stream(
    {"messages": [{"role": "user", "content": "What is 13 times 8?"}]},
    stream_mode="values",      # each chunk is the full state at that step
):
    latest = chunk["messages"][-1]
    if isinstance(latest, AIMessage) and latest.tool_calls:
        print("Calling tools:", [tc["name"] for tc in latest.tool_calls])
    elif isinstance(latest, AIMessage) and latest.content:
        print("Agent:", latest.content)

```

(`stream_mode="updates"` instead yields only what changed at each step — useful for logging.)

6.6 Giving the agent memory

Agents use a **checkpointer** (not `RunnableWithMessageHistory`) for memory. State is keyed by `thread_id`, the agent-world analogue of a chain's `session_id`.

```

from langgraph.checkpoint.memory import InMemorySaver

agent = create_agent(
    model="google_genai:gemin-2.5-flash",
    tools=[calculator, get_word_length],
    checkpointer=InMemorySaver(),      # persists state between invocations
)

cfg = {"configurable": {"thread_id": "session-1"}}
agent.invoke({"messages": [{"role": "user", "content": "My favourite number is 7."}]}, cfg)
agent.invoke({"messages": [{"role": "user", "content": "Multiply my favourite number by 6."}]}, cfg)
# -> remembers 7 and answers 42

```

In production, swap `InMemorySaver` for a persistent checkpointer (e.g. a Postgres-backed saver) so memory survives restarts.

6.7 When NOT to use an agent

Agents are powerful but costly and non-deterministic: each loop is another LLM round-trip, control flow can wander, and tool execution is a security surface (a model that can run `eval` or hit your database needs guardrails). If a task can be expressed as a fixed sequence, **use a chain** — it is cheaper, faster, and predictable. Reach for an agent only when the steps genuinely cannot be known in advance.

Historical context: Older code uses `initialize_agent`, `AgentExecutor`, and `create_react_agent` (the prompt-hub variety). The LangGraph-based `create_agent` supersedes them, adding durable execution, native streaming, checkpointer memory, and human-in-the-loop support.

Interview Questions — Agents & Tools

1. **What fundamentally distinguishes an agent from a chain?** (A chain runs a fixed sequence; an agent dynamically decides which tools to call, with what arguments, and in what order, looping until done.)
2. **Describe the agent reasoning loop.** (Each iteration rebuilds context — system prompt + tool schemas + message history — and asks the model for the next move: a final answer or tool call(s); tool results are appended and the loop repeats.)
3. **Why are a tool's docstring and type hints so important?** (The docstring tells the model when to use the tool; the type hints/schema define the arguments the model must produce — both directly drive correct tool selection and calls.)
4. **How do you inspect what an agent actually did?** (Examine the returned `messages` trace — tool calls, tool results, and the final answer — or stream intermediate steps.)
5. **What is a checkpointer, and how does it relate to chain memory?** (It persists agent state across turns keyed by `thread_id` — the agent equivalent of `RunnableWithMessageHistory`'s `session_id`.)
6. **Give two reasons an agent is riskier or costlier than a chain in production.** (Multiple LLM round-trips and non-deterministic control flow that can loop; plus the security surface of executing tools.)
7. **What does `create_agent` run on under the hood, and why does it matter?** (LangGraph, which provides durable execution, streaming, checkpointer memory, and human-in-the-loop — replacing the brittle legacy executors.)
8. **When should you deliberately choose a chain over an agent?** (Whenever the steps are known in advance — a chain is cheaper, faster, and deterministic.)

7. Conclusion — From Prototype to Production

You now have the full modern LangChain toolkit: **models and prompts**, **LCEL composition**, **memory**, **RAG**, and **agents**. Turning a working prototype into a dependable service rests on a few disciplines.

7.1 Make everything observable with LangSmith

LLM applications fail in quiet, non-obvious ways: a retriever returns irrelevant chunks, a prompt is subtly malformed, an agent loops.

LangSmith traces every step of every chain and agent — inputs, outputs, latency, token cost, and each tool call — with no code changes beyond two environment variables:

```
export LANGSMITH_TRACING=true
export LANGSMITH_API_KEY="your-langsmith-key"
```

You cannot improve what you cannot see; tracing is the highest-leverage production investment you can make.

7.2 Make outputs structured and reliable

Free text is hard to consume programmatically. Force the model to return validated, typed data with **structured output**:

```
from pydantic import BaseModel, Field

class Sentiment(BaseModel):
    label: str = Field(description="one of: positive, negative, neutral")
    confidence: float = Field(description="0.0 to 1.0")

structured = model.with_structured_output(Sentiment)
out = structured.invoke("I absolutely love this product!")
print(out.label, out.confidence)  # a parsed Pydantic object, not a string
```

Pair this with the resilience primitives from chapter 3 — `.with_retry()` for transient errors and `.with_fallbacks([...])` so one provider outage doesn't take the whole app down.

7.3 Control cost, latency, and quality

- **Right-size models.** Use a fast, cheap model (Gemini Flash, Llama 3.1) for routine work and reserve frontier models for genuinely hard reasoning. Ollama gives you zero marginal cost for development and many production workloads.
- **Stream and cache.** Stream responses so users see output immediately, and cache repeated identical calls to cut cost and latency.
- **Evaluate systematically.** Build a dataset of representative inputs and expected behaviours, and run it (LangSmith has evaluators built in) on every change. "It worked when I tried it once" is not a release criterion — regressions in LLM apps are easy to introduce and hard to spot by eye.

- **Guard the security surface.** Treat all model output as untrusted: validate tool arguments, sandbox anything that executes code, scope database/API credentials narrowly, and defend against prompt injection — especially in RAG, where retrieved documents can contain adversarial instructions.

7.4 The throughline

The single idea unifying modern LangChain is **composability**: small, well-understood Runnables piped together, observable end to end, and swappable between providers. Start with one LCEL chain. Add memory when the conversation needs it, retrieval when answers must be grounded in your data, and agents only when control flow genuinely cannot be fixed in advance. Build incrementally, measure everything, and keep prompts and tools small, typed, and well-documented. That discipline is what carries a project from a clever demo to something you can put in front of real users.